

Function block library

Modbus_TCP_2

for PC Worx

Documentation for
PHOENIX CONTACT function blocks
PHOENIX CONTACT GmbH Co. KG
Flachsmarktstrasse 8
D-32825 Blomberg, Germany

This documentation is available in English only.

Table of Contents

- [1 Installation hint](#)
- [2 General information](#)
 - [2.1 Modbus](#)
 - [2.2 Modbus TCP](#)
- [3 Change notes](#)
- [4 Supported PLCs](#)
- [5 Function blocks](#)
- [6 MB_TCP_Server](#)
 - [6.1 Function block call](#)
 - [6.2 Input parameters](#)
 - [6.3 Output parameters](#)
 - [6.4 Inout parameters](#)
 - [6.5 Diagnosis](#)
- [7 MB_TCP_Client](#)
 - [7.1 Function block call](#)
 - [7.2 Input parameters](#)
 - [7.3 Output parameters](#)
 - [7.4 Inout parameters](#)
 - [7.5 Diagnosis](#)
- [8 MB_TCP_FC1](#)
 - [8.1 Function block call](#)
 - [8.2 Input parameters](#)
 - [8.3 Output parameters](#)
 - [8.4 Inout parameters](#)
 - [8.5 Diagnosis](#)
- [9 MB_TCP_FC2](#)
 - [9.1 Function block call](#)
 - [9.2 Input parameters](#)
 - [9.3 Output parameters](#)
 - [9.4 Inout parameters](#)
 - [9.5 Diagnosis](#)
- [10 MB_TCP_FC3](#)
 - [10.1 Function block call](#)
 - [10.2 Input parameters](#)
 - [10.3 Output parameters](#)
 - [10.4 Inout parameters](#)
 - [10.5 Diagnosis](#)
- [11 MB_TCP_FC4](#)
 - [11.1 Function block call](#)
 - [11.2 Input parameters](#)
 - [11.3 Output parameters](#)
 - [11.4 Inout parameters](#)
 - [11.5 Diagnosis](#)

- [12 MB_TCP_FC5](#)
 - [12.1 Function block call](#)
 - [12.2 Input parameters](#)
 - [12.3 Output parameters](#)
 - [12.4 Inout parameters](#)
 - [12.5 Diagnosis](#)
- [13 MB_TCP_FC6](#)
 - [13.1 Function block call](#)
 - [13.2 Input parameters](#)
 - [13.3 Output parameters](#)
 - [13.4 Inout parameters](#)
 - [13.5 Diagnosis](#)
- [14 MB_TCP_FC15](#)
 - [14.1 Function block call](#)
 - [14.2 Input parameters](#)
 - [14.3 Output parameters](#)
 - [14.4 Inout parameters](#)
 - [14.5 Diagnosis](#)
- [15 MB_TCP_FC16](#)
 - [15.1 Function block call](#)
 - [15.2 Input parameters](#)
 - [15.3 Output parameters](#)
 - [15.4 Inout parameters](#)
 - [15.5 Diagnosis](#)
- [16 MB_TCP_DiagInfo_EN](#)
 - [16.1 Function block call](#)
 - [16.2 Input parameters](#)
 - [16.3 Output parameters](#)
 - [16.4 Inout parameters](#)
 - [16.5 Diagnosis](#)
- [17 Startup examples](#)
 - [17.1 System](#)
 - [17.2 Modbus client example](#)
 - [17.3 Block connection](#)
 - [17.4 Programm startup](#)
 - [17.5 Modbus client example](#)
 - [17.6 Block connection](#)
 - [17.7 Programm startup](#)
- [18 Appendix](#)
 - [18.1 Call Modbus data with associated function codes](#)
 - [18.2 IP_CONNECT, IP_URCV, IP_USEND](#)
 - [18.3 Data types](#)
- [19 Support](#)

1 Installation hint

If you did not specify a different directory during **library** installation all data in the MSI file will be unpacked to
c:\Users\Public\Documents\Phoenix Contact Libraries\PC Worx 6

Please copy the library data to your PC Worx 6 working library directory.

If you did not specify a different directory during **PC Worx 6** installation the default PC Worx 6 working library directory is

c:\Users\Public\Documents\PC WORX\Libraries

2 General information

2.1 Modbus

Modbus is an open protocol for control communication, in which various functions for read and write access to digital inputs and outputs and to registers are defined. Modbus is maintained by a dedicated user organization; documents with a detailed protocol description are available from the organization's homepage at www.modbus.org. Depending on the transmission system, a distinction is made between the Modbus RTU, Modbus ASCII, and Modbus TCP protocols.

2.2 Modbus TCP

For the MB_TCP_Server function block, the registers used for Modbus correspond to the elements of the udtTCP_ComData array. Each WORD array element is to be considered as a 16-bit Modbus register. These registers can be accessed from the local user program, in which the Modbus server is embedded, as well as from a remote control system when a Modbus client is used. For this purpose the mentioned function codes are available.

3 Change notes

Library version	Library build	PC Worx version	Change notes	Supported PLCs
2	20200709	6.30.2907	MB_TCP_Server_2: Adapted uiOffsetInputRegister and uiOffsetHoldingRegister	Refer to "Supported PLCs"
1	20200304	6.10.200	Update of documentation	"
1	20180316	6.10.200	Extracted from Modbus_3 library. Modbus_TCP_1 is NOT compatible with the Modbus TCP function blocks of Modbus_3. MB_TCP_Server_1: - Input tReconnectDelay: new. In former versions the value has been set to 100ms inside the function block. Now it can be set by the application. - Output udtDiag: new MB_TCP_Client_1: - Input tReconnectDelay: new In former versions the value has been set to 100ms inside the function block. Now it can be set by the application. - Output udtDiag: new MB_TCP_FC*_1: - wDiagCode in case of Modbus error is 16#C060, not 16#C050 any more because 16#C050 is used for IP_URCV error. This was a conflict in former versions.	"

4 Supported PLCs

AXC 1050 (2700988)
AXC 1050 XC (2701295)
AXC 3050 (2700989)
ILC 130 ETH (2988803)
ILC 131 ETH (2700973)
ILC 131 ETH/XC (2701034)
ILC 150 ETH (2985330)
ILC 151 ETH (2700974)
ILC 151 GSM/GPRS (2700977)
ILC 170 ETH 2TX (2916532)
ILC 171 ETH 2TX (2700975)
ILC 190 ETH 2TX (2700527)
ILC 191 ETH 2TX (2700976)
ILC 191 ME/AN (2700074)
ILC 191 ME/INC (2700075)
ILC 330 ETH (2737193)
ILC 330 PN (2988191)
ILC 350 ETH (2737203)
ILC 350 ETH/M (2985819)
ILC 350 PN (2876928)
ILC 370 ETH 2TX-IB (2876999)
ILC 370 PN 2TX-IB (2876915)
ILC 370 PN 2TX-IB/M (2985576)
ILC 390 PN 2TX-IB (2985314)
RFC 430 ETH-IB (2730190)
RFC 450 ETH-IB (2730200)
RFC 470S PN 3TX (2916794)
RFC 470 PN 3TX (2916600)
RFC 4072S (1051328)
RFC 480S PN 4TX (2404577)

5 Function blocks

Function block	Description	Version	Supported articles	License
MB_TCP_Client	The function block enables communication as client with Modbus TCP devices (server).	1	-	none
MB_TCP_Server	The function block enables communication as a server with a TCP client.	2	-	none
MB_TCP_DiagInfo_EN	This optional function block displays diagnostic messages of the Modbus client as plain text in English.	1	-	none
MB_TCP_FC*	The library contains function blocks for function codes 1, 2, 3, 4, 5, 6, 15, and 16.	1	-	none

6 MB_TCP_Server

This function block organizes the read and writes accesses of a remote control system to internal data via Modbus TCP. The data is stored in the arrModbusData block parameter, which is a WORD array with 7167 elements. Since this array has been defined as IN/OUT parameter, it can be read and written to in the local user program as required.

Attention: Depending on the control system used, the number of functions that the Modbus server can process in a given time is limited. In the event of overloading it may happen that the server does not respond to a telegram. In this case the accessing control system must send the telegram again.

Only one Modbus client at a time can access the Modbus server.

Access to Registers:

The Read Holding Registers, Write Multiple Registers and Read/Write Multiple Registers Modbus functions access the WORD elements in the udt_TCP_ComData array. The array index and the register address to be specified when calling the Modbus functions are identical.

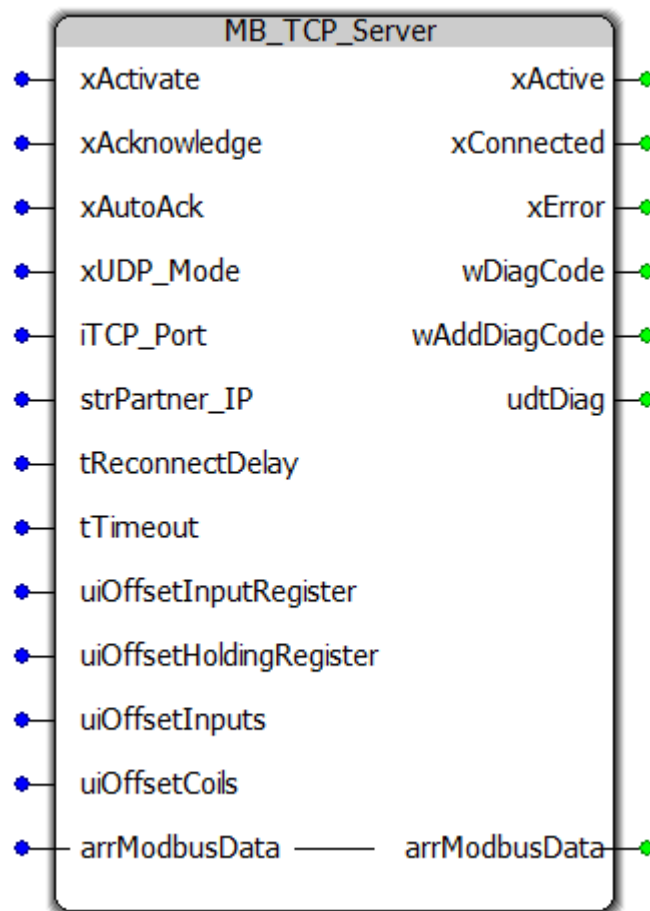
Access to Internal Bits:

When accessing internal bits all bits existing in the udt_TCP_ComData array are numbered consecutively. These numbers can be used to read this data with the Read Coils service. The numbering scheme is shown in below. The Modbus bit number must be specified when the data is accessed via the Modbus function.

	MODBUS_DATA[0] (WORD Data Type)															
	Byte 1								Byte 0							
Bit number	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Modbus Bit number	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

	MODBUS_DATA[1] (WORD Data Type)															
	Byte 1								Byte 0							
Bit number	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Modbus Bit number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16

6.1 Function block call



6.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
xAutoAck	BOOL	Automatic acknowledgment of errors TRUE: The error is acknowledged automatically. Error data is present at the xError, wDiagCode and wAddDiagCode outputs for only one cycle.
xUDP_Mode	BOOL	Selection of communication protocol. FALSE: Modbus-TCP is used TRUE: Modbus-UDP is used
iTCP_Port	INT	TCP port to be used. If no number is specified here, the default port 502 is used.
strPartner_IP	STRING	IP-Address of the client who is allowed to connect to the server. This function is necessary if multiple clients and servers are using the same TCP port. If no IP is specified here, all clients are allowed to connect to the server.
tReconnectDelay	TIME	Delay between two IP connect executions. Needed by PLC. Default value = 100ms.

tTimeout	Time	Timeout monitoring when there is a write access. After a write function was executed, a write telegram must be received again within the time specified here. A timeout error is reported if this is not the case. Specify the timeout value T#0s to disable this function. The following variables are used to define an offset for different data access starting with element 0 of arrModbusData. The offset has no effect on the element address.
uiOffsetInputRegister	UINT	Virtual address of the first input register. Example: The value 1000 means that the first Input-Register has the address 1000. Required for FC4.
uiOffsetHoldingRegister	UINT	Virtual address of the first holding register. Example: The value 1000 means that the first Holding-Register has the address 1000. Required for FC3, FC6, FC16.
uiOffsetInputs	UINT	Register-address of the first Input (0 – 7167). Required for FC2.
uiOffsetCoils	UINT	Register-address of the first Coil (0 – 7167). Required for FC1, FC5, FC15.

6.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xConnected	BOOL	TRUE: There is a connection to a Modbus client.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.
udtDiag	MB_TCP_UDT_SER_DIAG	Additional diagnostic data.

6.4 Inout parameters

Name	Type	Description
arrModbusData	MB_TCP_ARR_W_0_7167	WORD array First index: 0 Last index: 7167 This array contains the Modbus registers.

6.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active
16#8000	16#0000	The block is active
16#C010		Wrong Parameter
	16#0010	Invalid parameter at uiOffsetInputs (> 58368)
	16#0020	Invalid parameter at uiOffsetCoils (> 58368)
	16#0030	Invalid parameter at uiOffsetHoldingRegister (> 58368)
	16#0040	Invalid parameter at uiOffsetInputRegister. (> 58368)
16#C020	16#0000	Timeout error. A write services was executed and another write telegram was not received within the time specified in the tTimeout parameter. Enter the value T#0s in the tTimeout parameter to disable this function.
16#C030	16#xxxx	IP_CONNECT error 16#xxxx. Refer to appendix.
16#C040	16#xxxx	IP_USEND error 16#xxxx. Refer to appendix.
16#C050	16#xxxx	IP_URCV error 16#xxxx. Refer to appendix.

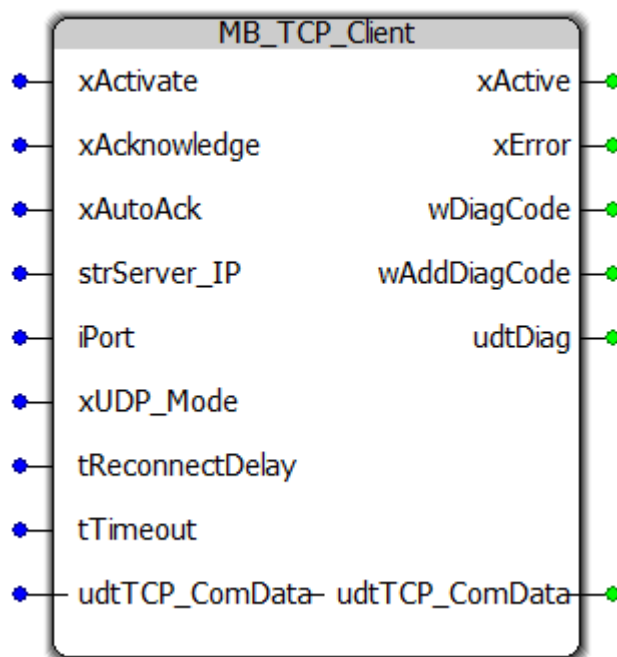
6.5.1 udtDiag

The udtDiag structure contains all inputs and outputs of the used firmware function blocks:

udtDiag	
udtIP_CONNECT	
xEN_C	TRUE
strPartner	/ACTIVE /PORT=502 /IP=10.250.65.20
xValid	TRUE
xError	FALSE
iStatus	4
iID	9
udtIP_USEND	
xReq	TRUE
iID	9
strR_ID	
xDone	FALSE
xError	FALSE
iStatus	2
udtIP_URCV	
xEN_R	TRUE
iID	9
strR_ID	
xNDR	FALSE
xError	FALSE
iStatus	1

7 MB_TCP_Client

7.1 Function block call



7.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
xAutoAck	BOOL	Automatic acknowledgment of errors TRUE: The error is acknowledged automatically. Error data is present at the xError, wDiagCode and wAddDiagCode outputs for only one cycle.
strServer_IP	STRING	IP address of the Modbus server. The address should be specified as follows: xxx.xxx.xxx.xxx
iPort	INT	TCP port of the Modbus server. If no address is specified here, the default address 502 is used.
xUDP_Mode	BOOL	FALSE: Modbus-TCP is used. TRUE: Modbus-UDP is used.
tReconnectDelay	TIME	Delay between two IP connect executions. Needed by PLC. Default value = 100ms.
tTimeout	TIME	Timeout for communication monitoring. The Modbus server must respond to a request within the time specified here, otherwise an error is triggered. If no value is specified here, the block operates with a timeout of 2 seconds.

7.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.
udtDiag	MB_TCP_UDT_CLI_DIAG	Additional diagnostic data.

7.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

7.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active
16#8000	16#0000	The block is active
16#C010		A parameterization error has occurred
	16#0010	The IP address of the Modbus server was not specified.
16#C020	16#0000	Timeout error
16#C030	16#xxxx	IP_CONNECT error 16#xxxx. Refer to appendix.
16#C040	16#xxxx	IP_USEND error 16#xxxx. Refer to appendix.
16#C050	16#xxxx	IP_URCV error 16#xxxx. Refer to appendix.

7.5.1 udtDiag

The udtDiag structure contains all inputs and outputs of the used firmware function blocks:

[-] udtDiag	
[-] udtIP_CONNECT	
xEN_C	TRUE
strPartner	/ACTIVE /PORT=502 /IP=10.250.65.20
xValid	TRUE
xError	FALSE
iStatus	4
iID	9
[-] udtIP_USEND	
xReq	TRUE
iID	9
strR_ID	
xDone	FALSE
xError	FALSE
iStatus	2
[-] udtIP_URCV	
xEN_R	TRUE
iID	9
strR_ID	
xNDR	FALSE
xError	FALSE
iStatus	1

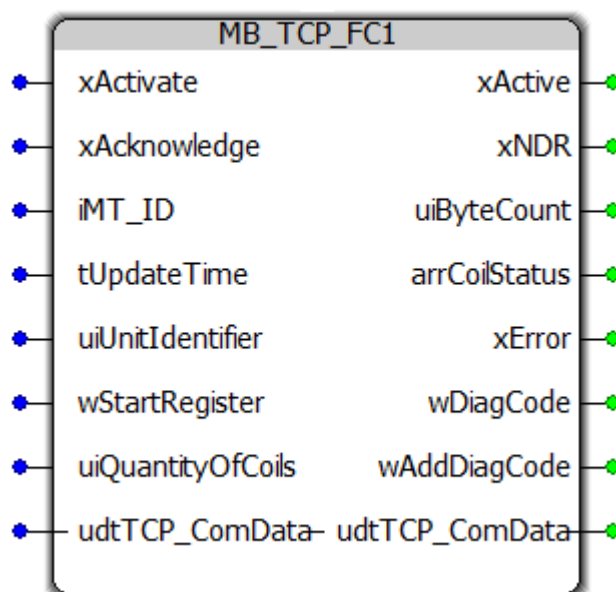
8 MB_TCP_FC1

This function block reads digital inputs from a Modbus slave.

The data is read in the form of a byte array (CoilStatus parameter), the first bit in the first byte of this array corresponds to the first output to be read. The following diagram shows the assignment of the bits in the byte array to the outputs and the internal bits, in this example output 1 to 16 are read:

	Byte 1 (Coil Status)								Byte 2 (Coil Status)							
Bit number	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Bit number	8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9

8.1 Function block call



8.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartRegister	WORD	Address of the first bit / output / register to be read.
uiQuantityOfCoils	UINT	Number of bits to be read. Valid value range: 1 - 2000.

8.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xNDR	BOOL	TRUE for one cycle: New data has been received.
uiByteCount	UINT	Number of bytes received.
arrCoilStatus	MB_TCP_ARR_B_1_260	Status of the outputs/internal bits. BYTE array with the limits 1 to 250. The first output corresponds to the first bit of the first byte.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

8.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

8.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

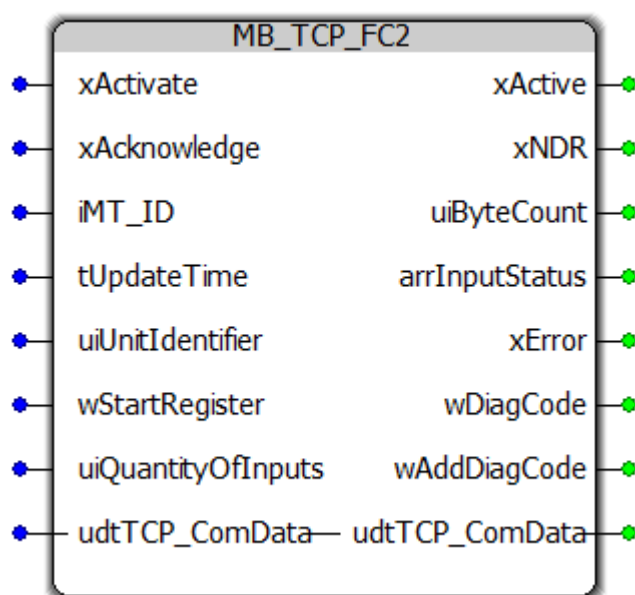
9 MB_TCP_FC2

This function block reads digital inputs from a Modbus slave.

The data is read in the form of a byte array (InputStatus parameter), the first bit in the first byte of this array corresponds to the first input to be read. The following diagram shows the assignment of the bits in the byte array to the inputs in this example inputs 1 to 16 are read:

	Byte 1 (Input Status)								Byte 2 (Input Status)							
Bit number	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Bit number	8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9

9.1 Function block call



9.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartRegister	WORD	Address of the first bit / output / register to be read.
uiQuantityOfInputs	UINT	Quantity of inputs to read.

9.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xNDR	BOOL	TRUE for one cycle: New data has been received.
uiByteCount	UINT	Number of bytes received.
arrInputStatus	MB_TCP_ARR_B_1_250	Status of the inputs. BYTE array with the limits 1 to 250. Each input is represented by a bit, the first input corresponds to the first bit of the first byte.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

9.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

9.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

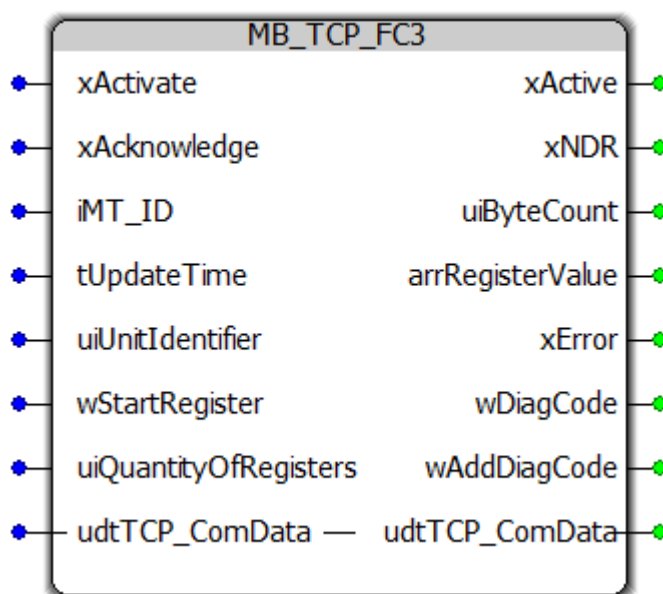
10 MB_TCP_FC3

This function block reads Holding Register from a Modbus slave.

The data is read in the form of a byte array (InputStatus parameter), the first bit in the first byte of this array corresponds to the first input to be read. The following diagram shows the assignment of the bits in the byte array to the inputs. In this example inputs 1 to 16 are read:

	Byte 1 (InputStatus)								Byte 2 (InputStatus)							
Bit number	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Bit number	8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9

10.1 Function block call



10.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartRegister	WORD	Address of the first bit / output / register to be read.
uiQuantityOfRegisters	UINT	Number of registers to be read. Valid value range: 1 - 125.

10.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xNDR	BOOL	TRUE for one cycle: New data has been received.
uiByteCount	UINT	Number of bytes received.
arrRegisterValue	MB_TCP_ARR_W_1_125	Values read from the registers. WORD array with the limits 1 to 125.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

10.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

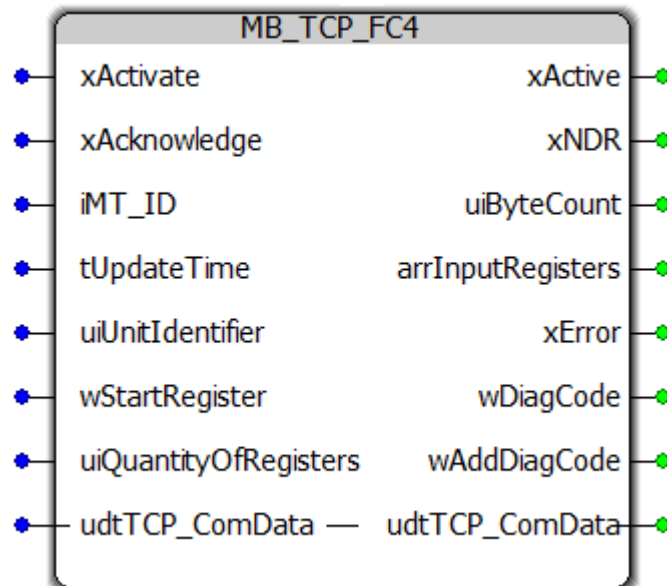
10.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

11 MB_TCP_FC4

This function block reads digital registers from a Modbus slave.

11.1 Function block call



11.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartRegister	WORD	Address of the first bit / output / register to be read.
uiQuantityOfRegisters	UINT	Number of registers to be read. Valid value range: 1 - 125.

11.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xNDR	BOOL	TRUE for one cycle: New data has been received.
uiByteCount	UINT	Number of bytes received.
arrInputRegisters	MB_TCP_ARR_W_1_125	Values read from the registers. WORD array with the limits 1 to 125.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

11.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

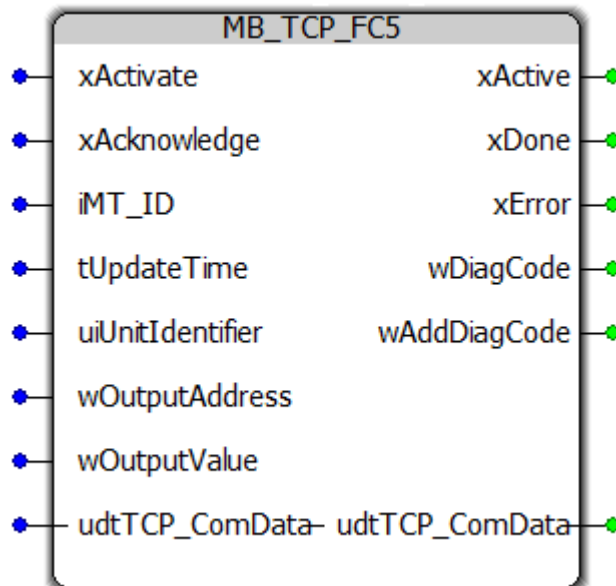
11.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

12 MB_TCP_FC5

This function block writes a bit in the memory of a Modbus slave.

12.1 Function block call



12.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wOutputAddress	WORD	Address of the bit/output to be written. During addressing, the address of the first digital output is zero.
wOutputValue	WORD	Value to be written. Only the following two values are permitted: TRUE: 16#FF00 FALSE: 16#0000

12.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xDone	BOOL	TRUE: Request was sent and the response from communication partner received successfully.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

12.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

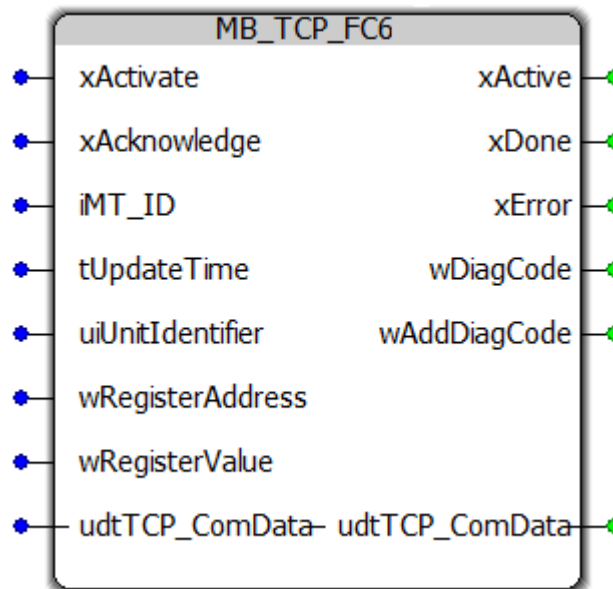
12.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

13 MB_TCP_FC6

This function block writes a register into the memory of a Modbus slave.

13.1 Function block call



13.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wRegisterAddress	WORD	Address of the register to be written.
wRegisterValue	WORD	Value to be written to the register.

13.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xDone	BOOL	TRUE: Request was sent and the response from communication partner received successfully.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

13.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

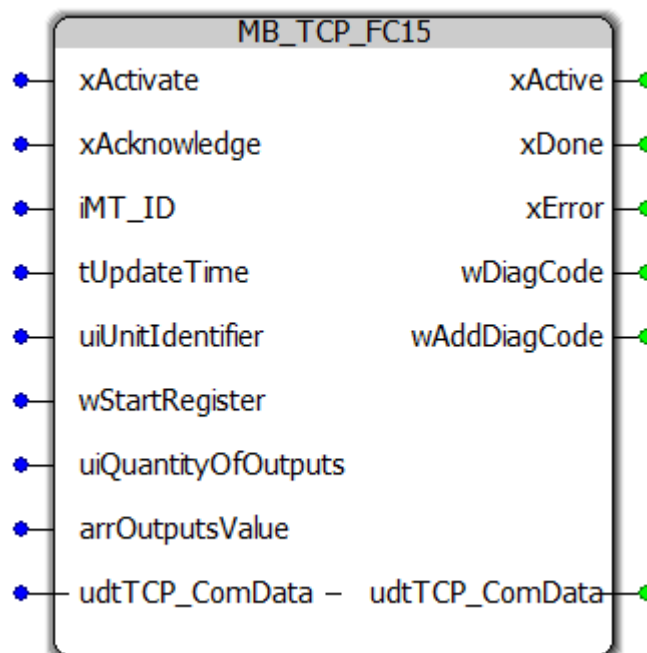
13.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

14 MB_TCP_FC15

This function block writes multiple bits into the memory of the Modbus slave. The data is read in the form of a byte array, the first bit in the first byte of this array corresponds to the first output to be written. The following diagram shows the assignment of the bits in the byte array to the outputs; in this example outputs 1 to 16 are written:

14.1 Function block call



14.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartRegister	WORD	Address of the first bit / output / register to be read.
uiQuantityOfOutputs	UINT	Number of outputs to be written. Valid value range: 1 - 1000.
arrOutputsValue		

14.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xDone	BOOL	TRUE: Request was sent and the response from communication partner received successfully.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

14.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

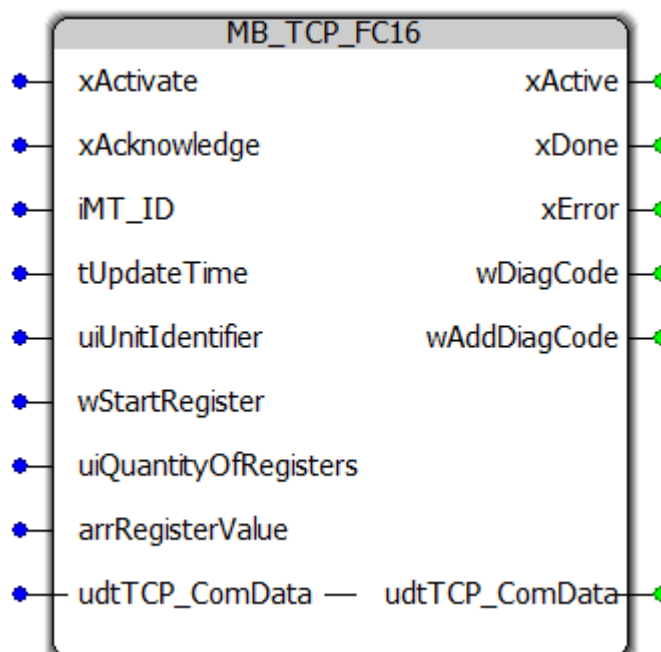
14.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

15 MB_TCP_FC16

This function block writes multiple registers into the memory of a Modbus slave.

15.1 Function block call



15.2 Input parameters

Name	Type	Description
xActivate	BOOL	Rising edge: Activates the function block. FALSE: Deactivates the function block.
xAcknowledge	BOOL	Rising edge: Error messages are deleted. The function block is not re-initialized.
iMT_ID	INT	Value in the range from 1 to 10. Represents the connection between the blocks and may only be assigned once per MB_TCP_Client. This parameter should be static.
tUpdateTime	TIME	Waiting time in ms between two read tasks.
uiUnitIdentifier	UINT	Parameter for addressing of a Modbus RTU device, which is connected via a gateway
wStartingAddress	WORD	Address of the first register to be written.
uiQuantityOfRegisters	UINT	Number of registers to be read. Valid value range: 1 - 125.
arrRegisterValue	MB_TCP_ARR_W_1_125	Values read from the registers. WORD array with the limits 1 to 125.

15.3 Output parameters

Name	Type	Description
xActive	BOOL	FALSE: Function block is not active. TRUE: Function block is active.
xDone	BOOL	TRUE: Request was sent and the response from communication partner received successfully.
xError	BOOL	TRUE: An error has occurred. For details refer to wDiagCode and wAddDiagCode.
wDiagCode	WORD	Diagnosis code. Refer to diagnostic table.
wAddDiagCode	WORD	Additional diagnosis code. Refer to diagnostic table.

15.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

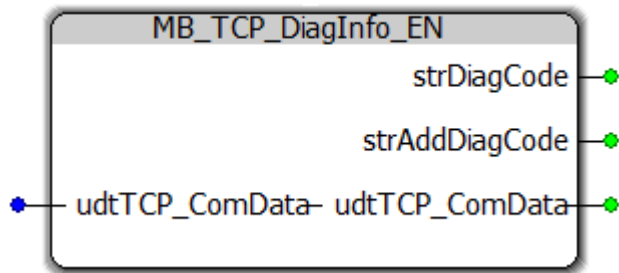
15.5 Diagnosis

wDiagCode	wAddDiagCode	Description
16#0000	16#0000	The block is not active.
16#8000	16#0000	The block is active.
16#C010	16#0000	The MB_TCP_Client block is not ready.
16#C030	16#0000	The iMT_ID used is invalid. The iMT_ID must be in the range from 1 to 10.
16#C040	16#0000	The iMT_ID used is assigned twice. Several Modbus function blocks use the same iMT_ID, in this case only the first called block operates, all other blocks indicate this error.
16#C060		Modbus error
	16#0001	Illegal function
	16#0002	Illegal data address
	16#0003	Illegal data value
	16#0004	Slave device failure
	16#0005	Acknowledge
	16#0006	Slave device busy
	16#0008	Memory parity error
	16#000A	Gateway path unavailable
	16#000B	Gateway target device failed to respond

16 MB_TCP_DiagInfo_EN

If there is an error, this block shows the diagnostics of the master block as a text in English. The source code of the block can be read and modified. To show the diagnostic messages in other languages, copy the block and translate the diagnostic text into the desired language. The text outputs (strDiagCode and strAddDiagCode) are limited to 80 characters.

16.1 Function block call



16.2 Input parameters

None

16.3 Output parameters

Name	Type	Description
strDiagCode	STRING	If there is an error, the variable shows the description for the current wDiagCode in English.
strAddDiagCode	STRING	If there is an error, the variable shows the description for the current wAddDiagCode in English.

16.4 Inout parameters

Name	Type	Description
udtTCP_ComData	MB_TCP_UDT_COMMUNICATION	Communication structure of the block family. The blocks are connected together using this structure.

16.5 Diagnosis

None

17 Startup examples

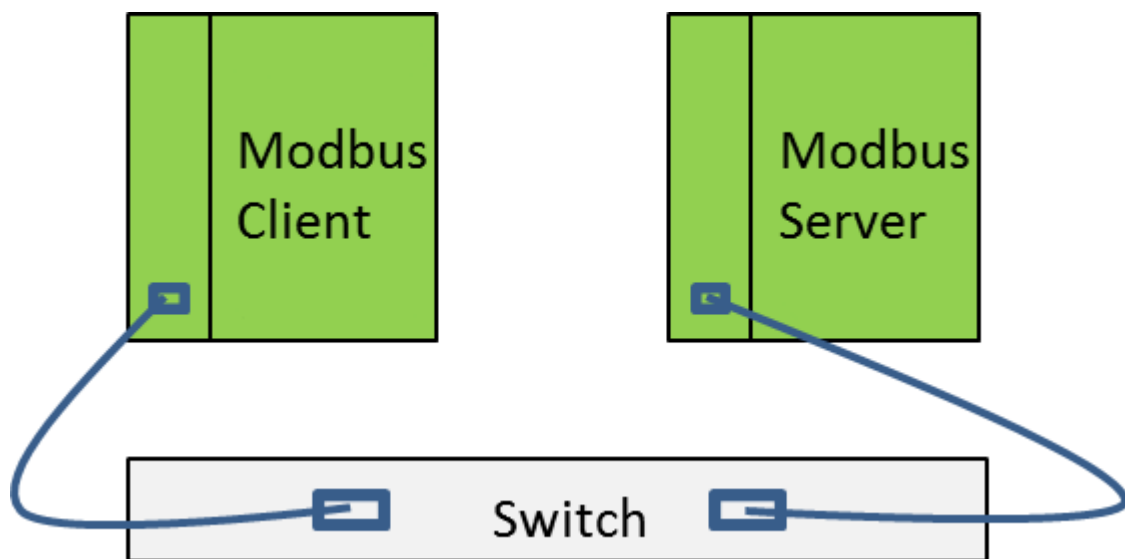
For the startup instruction of the Modbus_TCP function block library please find the following example:

- MB_TCP_2_EXA_Server.zwt
- MB_TCP_2_EXA_Client.zwt

This example is located in the “Examples” folder of the unzipped msi file of the library.

It describes the communication between a Modbus server and a Modbus client.

17.1 System

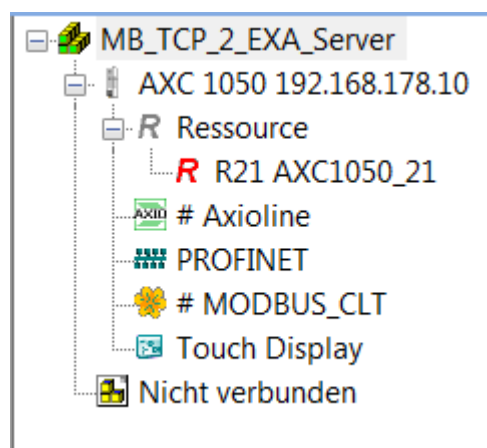


17.2 Modbus client example

17.2.1 Bus structure

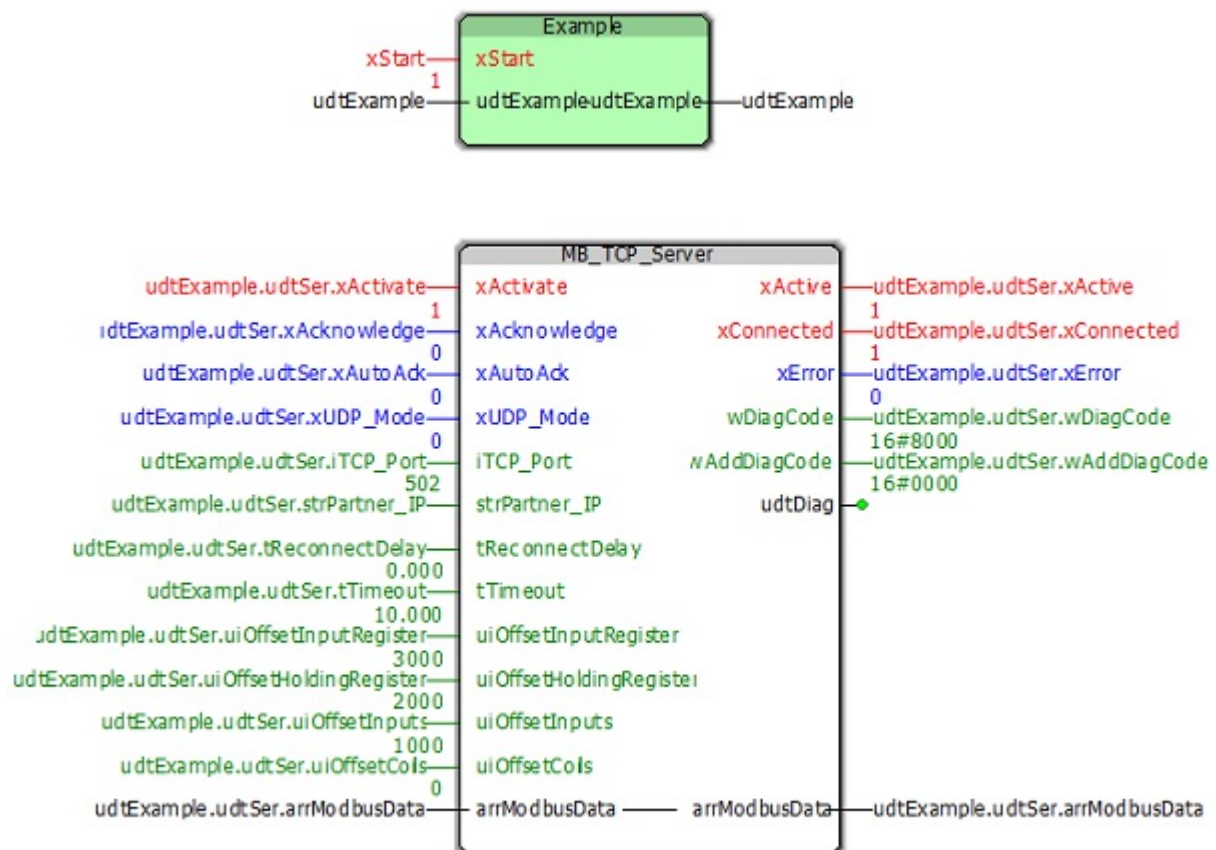
For this example, the following hardware is used:

- AXC 1050 (2700988)



17.3 Block connection

17.3.1 Main



17.4 Programm startup

```
CASE iState OF
0: (* Init TCP_Server for example and wait for xStart *)
    udtExample.udtSer.xActivate           := FALSE;
    udtExample.udtSer.xAcknowledge        := FALSE;
    udtExample.udtSer.xAutoAck            := FALSE;
    udtExample.udtSer.xUDP_Mode           := FALSE;
    udtExample.udtSer.iTCP_Port           := 502;
    udtExample.udtSer.strPartner_IP       := '';
    udtExample.udtSer.tTimeout            := TIME#10s;
    udtExample.udtSer.uiOffsetInputRegister := UINT#3000;
    udtExample.udtSer.uiOffsetHoldingRegister := UINT#2000;
    udtExample.udtSer.uiOffsetInputs      := UINT#1000;
    udtExample.udtSer.uiOffsetCoils       := UINT#0;
    FOR iIndex := 0 TO 7167 DO
        udtExample.udtSer.arrModbusData[iIndex] := INT_TO_WORD(iIndex);
    END FOR;
    udtExample.udtSer.arrModbusData[0] := WORD#16#FEDC;
    IF xStart = TRUE THEN
        iState := 100;
    END_IF;

100: (* *)
    udtExample.udtSer.xActivate := TRUE;
    IF udtExample.udtSer.xActive = TRUE THEN
        iState := 200;
    END_IF;

200: (* wait for xStart = FALSE *)
    IF xStart = FALSE THEN
        iState := 300;
    END_IF;

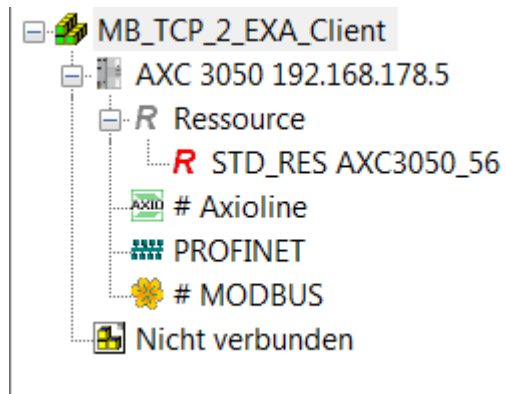
300: (* Deactivate TCP_Server *)
    udtExample.udtSer.xActivate := FALSE;
    IF udtExample.udtSer.xActive = FALSE THEN
        iState := 0;
    END_IF;
END_CASE;
```

17.5 Modbus client example

17.5.1 Bus structure

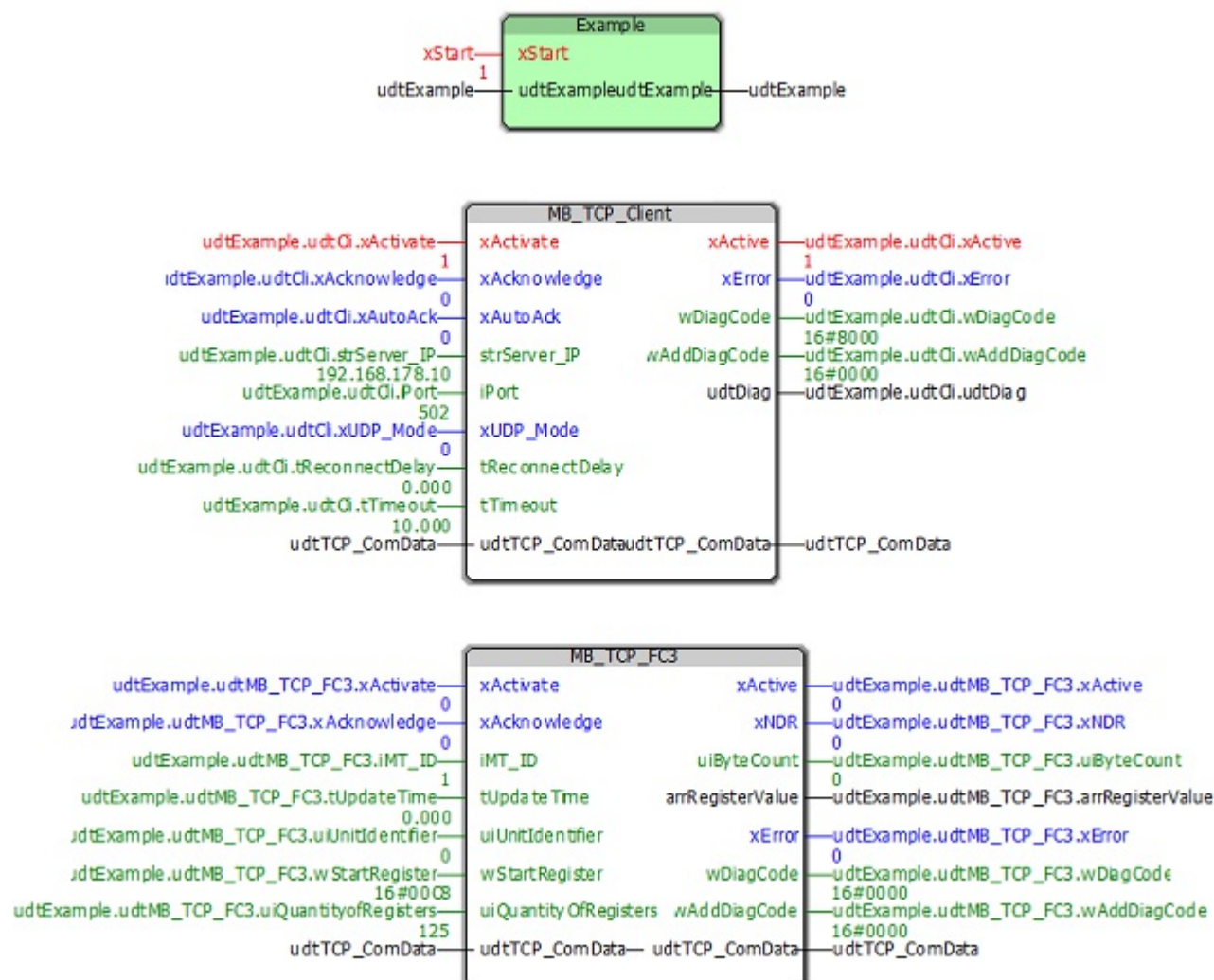
For this example, the following hardware is used:

- AXC 3050 (2700989)



17.6 Block connection

17.6.1 Main



17.7 Programm startup

CASE iState OF

```

0: (* Wait for start of Example program *)
  IF xStart = TRUE THEN
    iState := 10;
  END_IF;

10: (* Initialization of client and FC3 function block *)
  (* Client *)
  udtExample.udtCli.xActivate := FALSE;
  udtExample.udtCli.xAutoAck := FALSE;
  udtExample.udtCli.strServer_IP := '192.168.178.10';
  udtExample.udtCli.iPort := 502;
  udtExample.udtCli.xUDP_Mode := FALSE;
  udtExample.udtCli.tTimeout := TIME#10s;

  (* FC3 function block *)
  udtExample.udtMB_TCP_FC3.xActivate := FALSE;
  udtExample.udtMB_TCP_FC3.iMT_ID := 1;

  iState := 100;

100: (* Start client function block first and wait for xActive *)
  udtExample.udtCli.xActivate := TRUE;
  IF (udtExample.udtCli.xActive = TRUE) THEN
    iState := 1000;
  END_IF;

  (* Read min. number of registers *)

1000: (* Now activate FC3 function block and set inputs *)
  udtExample.udtMB_TCP_FC3.xActivate := TRUE;
  udtExample.udtMB_TCP_FC3.uiQuantityOfRegisters := UINT#1;
  (* udtFC3.arrRegisterValues = ARRAY [1..125] OF WORD *)
  udtExample.udtMB_TCP_FC3.wStartRegister := WORD#0;
  (* Wait for xActive of FC3 function block *)
  IF (udtExample.udtMB_TCP_FC3.xActive = TRUE) THEN
    iState := 1010;
  END_IF;

1010: (* Check values - the FC reads Register 2000 from the Modbus_TCP_Server*)
  (* Modbus server has to deliver these values! *)
  IF (udtExample.udtMB_TCP_FC3.xError = FALSE)
    AND (udtExample.udtMB_TCP_FC3.arrRegisterValue[1] = WORD#16#07D0)
  THEN
    iState := 2000;
  END_IF;

  (* Read max. number of registers *)

2000: (* Deactivation of FC3 function block before read again *)
  udtExample.udtMB_TCP_FC3.xActivate := FALSE;
  (* Wait for xActive = FALSE after deactivation *)
  IF (udtExample.udtMB_TCP_FC3.xActive = FALSE) THEN
    iState := 2010;
  END_IF;

2010: (* Activation of FC3 function block and set inputs *)
  udtExample.udtMB_TCP_FC3.xActivate := TRUE;
  (* udtFC3.arrRegisterValues = ARRAY [1..125] OF WORD *)
  udtExample.udtMB_TCP_FC3.uiQuantityOfRegisters := UINT#125;

```

```
(* read Register 2200 to 2324 *)
udtExample.udtMB_TCP_FC3.wStartRegister := WORD#200;
(* Wait for xActive = TRUE *)
IF
    udtExample.udtMB_TCP_FC3.xActive = TRUE AND
    udtExample.udtMB_TCP_FC3.xError = FALSE
THEN
    iState := 2020;
END_IF;

2020: (* Check values, if uiByteCount = 250 values are read correctly *)
    (* Modbus server has to deliver these values! *)
    IF
        udtExample.udtMB_TCP_FC3.xError = FALSE AND
        udtExample.udtMB_TCP_FC3.uiByteCount = UINT#250
    THEN
        iState := 2100;
    END_IF;
2100: (* Deactivate FC3 function block *)
    udtExample.udtMB_TCP_FC3.xActivate := FALSE;
    IF udtExample.udtMB_TCP_FC3.xActive = FALSE THEN
        iState := 2200;
    END_IF;
2200: (* wait for xStart = FALSE *)
    IF xStart = FALSE THEN
        iState := 3000;
    END_IF;
3000: (* Finish *)
    (* Deactivate client function block *)
    udtExample.udtCli.xActivate := FALSE;
    IF (
        udtExample.udtMB_TCP_FC3.xActive = FALSE
    ) THEN
        iState := 0;
    END_IF;
END_CASE;
```

18 Appendix

18.1 Call Modbus data with associated function codes

Data access		Functioncode	Offset	Start Address	Length	arrModbusData
(Array of 7167 Word)						
Bit	Discret Input	2 Read Discret Input	200	0	16	200
		2 Read Discret Input	200	16	16	201
Bit	Discrete Output (Coils)	1 Read Coils	300	0	16	300
		1 Read Coils	300	16	16	301
		5 Write Single Coils	300	0	1	300
		15 Write Multiple Coils	300	0	16	300
		15 Write Multiple Coils	300	16	16	301
Word	Input Register	4 Read Input Register	0	0	10	0-9
		4 Read Input Register	0	20	10	20-29
Word	Holding Register	3 Read Holding Register	100	100	10	0-9
		6 Write Singel Register	100	100	1	0
		16 Write Multiple register	100	100	10	0-9

18.2 IP_CONNECT, IP_URCV, IP_USEND

Diagnosis of firmware function blocks

Error code	Description	Status only for
16#0001	Creation of at least one task was unsuccessful.	
16#0002	Initialization of the socket interface unsuccessful (Windows NT only).	
16#0003	Dynamic memory could not be reserved.	IP_USEND, IP_URCV
16#0004	FB cannot be initialized because an error occurred during startup of the asynchronous communication task.	IP_USEND, IP_URCV, IP_TIME, IP_CONNECT
16#0010	Socket initialization unsuccessful.	
16#0011	Error while transmitting a message.	
16#0012	Error while receiving a message.	
16#0013	Unknown service code in the telegram header.	
16#0021	Invalid condition switch while establishing a link.	IP_CONNECT
16#0030	There are no more free channels available.	IP_CONNECT
16#0031	Connection has been terminated.	IP_CONNECT
16#0033	General timeout, receiver is no longer answering or the transmitter has not finished transmitting.	IP_USEND, IP_URCV
16#0034	Request for connection was denied.	IP_CONNECT
16#0035	Receiver has not confirmed transmission, receiver may be overloaded (repeat transmission).	IP_USEND
16#0040	Partner string is too long (max. 255 characters).	IP_CONNECT, IP_TIME
16#0041	The specified IP address is not valid or could not be interpreted correctly.	IP_CONNECT, IP_TIME
16#0042	Invalid port number.	IP_CONNECT
16#0044	Wrong digit in some part of the PARTNER string.	IP_CONNECT
16#0045	Unknown parameter at input "PARTNER" (maybe upper/lower case or misspelling).	IP_CONNECT, IP_TIME
16#0046	Missing parameter, parameter string is incomplete.	IP_CONNECT
16#0047	Missing port while using a serve (/PASSIVE).	IP_CONNECT
16#0048	Invalid type. Cannot find any of the allowed connection types.	IP_CONNECT
16#0050	Attempt to transmit using invalid link (transmitter or receiver).	IP_USEND
16#0053	All available connections are used.	
16#0061	Neg. confirmation by receiver. An incorrect sequence number has been used.	IP_USEND, IP_URCV
16#0062	Data types of transmitter and receiver do not match.	IP_USEND, IP_URCV
16#0063	Receiver is not yet ready to receive (possible cause: Receiver is disabled or is currently in transfer mode (NDR=TRUE)).	IP_USEND
16#0064	No receiver block with the matching R_ID was found.	IP_USEND
16#0065	Another block instance already uses this connection.	
16#0070	Partner control system was not configured as the time server.	

18.3 Data types

TYPE

```

(* Byte arrays *)

MB_TCP_ARR_B_1_250 : ARRAY [1..260] OF BYTE;

(* Word arrays *)

MB_TCP_ARR_W_1_125 : ARRAY [1..125] OF WORD;
MB_TCP_ARR_W_0_7167 : ARRAY [0..7167] OF WORD;

MB_TCP_UDT_WINSOCK : STRUCT
    DataCnt          : WORD;
    Transaction_ID   : WORD;
    Protocol_ID      : WORD;
    Length           : WORD;
    arrData : MB_TCP_ARR_B_1_250;
END_STRUCT;

MB_TCP_UDT_READ_WRITE : STRUCT
    Write              : BOOL; (* FALSE = Read, TRUE = Write *)
    xActive            : BOOL; (* TRUE: Reading data *)
    FC                 : INT;  (* Modbus function code *)
    uiUnitIdentifier   : BYTE; (* Unit identifier *)
    (* Read *)
    ReadStartAddress   : WORD; (* Address of the first register *)
    QuantityToRead     : WORD; (* Number of registers *)
    arrReadBuffer      : MB_TCP_ARR_B_1_250; (* Read buffer *)
    ByteCount          : BYTE; (* Number of received bytes *)
    xNDR               : BOOL; (* TRUE (for one cycle): New data available *)
    (* Write *)
    WriteStartAddress   : WORD; (* Address of first register *)
    QuantityToWrite     : WORD; (* Number of registers *)
    arrWriteBuffer      : MB_TCP_ARR_B_1_250; (* Write buffer *)
    xDone              : BOOL; (* TRUE (for one cycle): Write is finished *)
    (* Diagnosis *)
    xError              : BOOL; (* TRUE: Error *)
    wDiagCode           : WORD; (* Diag code *)
    wAddDiagCode        : WORD; (* Additional diag code *)
    (* Internal *)
    StartAction         : BOOL; (* Used by scheduler -> TRUE = Read data *)
    tUpdateTime         : Time; (* Time between two read operations in ms *)
END_STRUCT;

MB_TCP_ARR_READ_WRITE : ARRAY [1..10] OF MB_TCP_UDT_READ_WRITE;

MB_TCP_UDT_COMMUNICATION : STRUCT
    ClientReady        : BOOL;
    xError              : BOOL;
    wDiagCode           : WORD;
    wAddDiagCode        : WORD;
    ReadWriteGroups    : MB_TCP_ARR_READ_WRITE;
END_STRUCT;

MB_TCP_UDT_MBS_REQUEST : STRUCT
    DATA_CNT:        WORD; (* Not part of Modbus protocol *)
    Transaction_ID:    WORD;
    Protocol_ID:       WORD;
    Length:            WORD;
    Unit_ID:           BYTE;
    FC:                BYTE;

```

```

        Reference:      WORD;
        Word_Count:     WORD;
        DATA:          MB_TCP_ARR_B_1_250;
    END_STRUCT;

    MB_TCP_UDT_MBS_RESPONSE : STRUCT
        DATA_CNT:       WORD; (* Not part of Modbus protocol *)
        Transaction_ID:  WORD;
        Protocol_ID:     WORD;
        Length:          WORD;
        Unit_ID:         BYTE;
        FC:              BYTE;
        DATA:           MB_TCP_ARR_B_1_250;
    END_STRUCT;

    (* Buffers for requests *)
    MB_TCP_ARR_MBS_BUFFER: ARRAY [0..10] OF MB_TCP_UDT_MBS_REQUEST;

    (* I/O parameter of firmware function blocks as struct *)

    MB_TCP_UDT_IP_CONNECT : STRUCT
        (* Inputs *)
        xEN_C      : BOOL;
        strPartner : STRING;
        (* Outputs *)
        xValid      : BOOL;
        xError      : BOOL;
        iStatus     : INT;
        iID         : INT;
    END_STRUCT;

    MB_TCP_UDT_IP_USEND : STRUCT
        (* Inputs *)
        xReq      : BOOL;
        iID       : INT;
        strR_ID   : STRING;
        (* Outputs *)
        xDone     : BOOL;
        xError    : BOOL;
        iStatus   : INT;
    END_STRUCT;

    MB_TCP_UDT_IP_URCV : STRUCT
        (* Inputs *)
        xEN_R      : BOOL;
        iID        : INT;
        strR_ID    : STRING;
        (* Outputs *)
        xNDR       : BOOL;
        xError     : BOOL;
        iStatus    : INT;
    END_STRUCT;

    (* Diag structure for MB_TCP_Server *)

    MB_TCP_UDT_SER_DIAG : STRUCT
        udtIP_CONNECT : MB_TCP_UDT_IP_CONNECT;
        udtIP_USEND   : MB_TCP_UDT_IP_USEND;
        udtIP_URCV    : MB_TCP_UDT_IP_URCV;
    END_STRUCT;

    (* Diag structure for MB_TCP_Client *)

    MB_TCP_UDT_CLI_DIAG : STRUCT

```

```
    udtIP_CONNECT    : MB_TCP_UDT_IP_CONNECT;  
    udtIP_USEEND     : MB_TCP_UDT_IP_USEEND;  
    udtIP_URCV       : MB_TCP_UDT_IP_URCV;  
END_STRUCT;  
  
END_TYPE
```

19 Support

For technical support please contact your local PHOENIX CONTACT agency
at <https://www.phoenixcontact.com>

Owner:

PHOENIX CONTACT Electronics GmbH
Business Unit Automation Systems
System Services
Library Services